SystemBuilder

Relatório de Progresso de Definição do Produto

Estado-base: v0.6

Data de referência: 23 de setembro de 2026

Fase: Definição de produto

Status em uma frase: o SystemBuilder já possui visão, escopo inicial, proposta de valor, princípios de experiência, fronteira funcional com o BMAD e baseline de State Model suficientemente definidos para avançar à formalização dos contratos de etapa e do Assurance Core; ainda não há Product Discovery concluído, engenharia realizada ou validações jurídica, de segurança e de mercado concluídas.

Sumário

1. Sumário executivo **1**

2. Objetivo do trabalho e deste documento **1**
2.1 Objetivo do trabalho de definição . . . 1
2.2 Objetivo deste relatório . . . 2
2.3 Convenção epistemológica . . . 2

3. Descrição atual do produto **2**
3.1 O que é o SystemBuilder . . . 2
3.2 Proposta de valor . . . 2
3.3 Ciclo de desenvolvimento suportado . . . 3
3.4 Ciclo de Coding por story . . . 3
3.5 Jornada principal provisória . . . 3
3.6 Escopo futuro, fora da primeira fase . . . 4

4. Usuários e modelo de uso **4**
4.1 Segmentação operacional . . . 4
4.2 Modo de interação e política de autonomia . . . 4
4.3 Modelo de colaboração do MVP . . . 5

5. Arquitetura funcional do produto **5**
5.1 Relação entre SystemBuilder e BMAD . . . 5
5.2 Responsabilidade do SB Control Plane . . . 5
5.3 Responsabilidade nativa do BMAD . . . 5
5.4 Adapter versionado / anti-corruption layer . . . 6
5.5 Política de atualização do BMAD . . . 6

6. Assurance, segurança e governança da autonomia **6**
6.1 Assurance como parte do produto . . . 6
6.2 Direções já definidas . . . 6
6.3 Classes de falha e risco consideradas . . . 7
6.4 Segurança, privacidade e compliance ainda abertos . . . 7

7. State Model atual **7**
7.1 Estado macro e estado local . . . 7
7.2 Vocabulário local baseline . . . 7
7.3 Contrato de saída como condição de conclusão . . . 8
7.4 Mudanças upstream e staleness . . . 8
7.5 Entrada intermediária: IMPORT_ADOPTION . . . 8

8. Capacidades e requisitos atuais **9**
8.1 Capacidades funcionais principais . . . 9
8.2 Requisitos não funcionais já estabelecidos . . . 9
8.3 Dados e integrações ainda em definição . . . 9

9. Histórico de evolução e decisões **10**
9.1 Evolução por versão do ESTADO . . . 10
9.2 Decisões confirmadas . . . 10

10. Parecer especializado BMAD e impacto **11**
10.1 Objetivo do handoff . . . 11
10.2 Conclusões incorporadas . . . 11

11. Pendências atuais e questões abertas **11**
11.1 Pendências efetivamente abertas . . . 12
11.2 Nota de integridade do ESTADO v0.6 . . . 12

12. Riscos, restrições e dependências **12**
12.1 Riscos registrados . . . 12
12.2 Restrições atuais . . . 13

13. Métricas e critérios de sucesso ainda em construção **13**

14. Próximo passo imediato — decisões pendentes **14**
14.1 Objeto seguinte: contrato genérico de uma etapa . . . 14
14.2 Decisão 1 — resultado do Readiness . . . 15
14.3 Decisão 2 — Override . . . 15
14.4 Decisão 3 — Assurance Core mínimo . . . 15
14.5 Decisão 4 — versionamento ao reabrir etapas . . . 16
14.6 Recomendação consolidada da rodada . . . 16

15. Próximos passos prováveis até a definição final **16**
15.1 Fechar as quatro decisões imediatas . . . 16
15.2 Produzir o Stage Contract v0.1 . . . 16
15.3 Aplicar o contrato a três etapas representativas . . . 17
15.4 Revisão adversarial intermediária . . . 17
15.5 Reconciliar e generalizar o modelo . . . 17
15.6 Definir modelo de dados funcional e artefatos . . . 17
15.7 Detalhar integração com IA e BMAD . . . 17
15.8 Segurança, privacidade, compliance e licenciamento . . . 18
15.9 UX e jornadas detalhadas . . . 18
15.10 Métricas, eventos e critérios de aceitação . . . 18
15.11 Gate de maturidade e revisão adversarial final . . . 18
15.12 PDD final . . . 18

16. Limites atuais da definição **18**

Anexo A — Registro completo de requisitos funcionais **19**

Anexo B — Registro epistemológico consolidado **20**

Anexo C — Pendências do ESTADO v0.6 e leitura reconciliada **21**

Anexo D — Fontes internas utilizadas **22**

1. Sumário executivo

O **SystemBuilder (SB)** está sendo definido como uma plataforma de orquestração guiada para desenvolvimento de software com IA. Seu objetivo é permitir que diferentes perfis de usuário avancem de uma ideia inicial até um sistema implantado, reduzindo curva de aprendizado e atrito operacional, enquanto o produto controla contexto, pré-requisitos, artefatos, decisões, estados, evidências e permissões relevantes ao longo do processo.

A primeira versão utilizável deverá cobrir o ciclo completo **ideia → deploy**, ainda que algumas etapas sejam inicialmente menos profundas. A experiência será própria do SB, e o processo será principalmente baseado em uma versão estabilizada e customizada do **BMAD**, mantida sob política própria de homologação e atualização.

Ao longo da definição, o conceito evoluiu de uma ferramenta principalmente voltada à simplificação do uso de IA e do BMAD para um produto com um segundo eixo de valor igualmente importante: **assurance, auditabilidade, rastreabilidade e governança da autonomia**. O SB deverá permitir responder, para ações relevantes, o que foi feito, por que foi feito, sobre qual estado, com quais permissões, com qual evidência e com qual risco residual.

Uma análise especializada do **BMad Method v6.12.0** concluiu que o BMAD já possui mecanismos relevantes de estado, persistência, artefatos, readiness e tracking, porém distribuídos por diferentes workflows e sem um ledger transversal que cubra a semântica de assurance pretendida pelo SB. A decisão adotada foi uma arquitetura funcional **native-first**: o BMAD permanece autoridade sobre seu conteúdo e estado nativos quando adequados; o SB mantém um **control plane transversal** para orquestração, identidade de execução, autonomia, gates, proveniência, evidências, risco residual e reconciliação, com um **adaptador versionado** entre o produto e cada versão homologada do BMAD.

Também foram fixados princípios estruturais para o MVP:

- cobertura funcional dos segmentos operacionais 1 a 4: usuário ocasional/não técnico, maker/indie, founder/PM/designer e desenvolvedor profissional;
- um usuário principal por projeto no MVP;
- modos de interação **Guiado** e **Avançado**, independentes da política de autonomia **Conservadora / Supervisionada / Autônoma**;
- default conceitual **Guiado + Supervisionada**;
- execução serial nos escopos em que exista risco de conflito, porém com modelo preparado para futura concorrência;
- **Assurance Core obrigatório desde Ideação/Brainstorm**, com profundidade crescente ao longo do ciclo;
- uma única etapa macro primária ativa por vez;
- conclusão de etapa condicionada ao respectivo contrato de saída;
- alterações upstream geram análise de impacto e `STALE/REQUIRES_REVIEW`, em vez de destruição automática do downstream;
- entrada intermediária no processo passa por um fluxo explícito de `IMPORT_ADOPTION`.

O trabalho chegou, portanto, ao ponto em que o próximo artefato estruturante deverá ser o **Stage Contract genérico do SB**, acompanhado da especificação do Assurance Core. Antes disso, quatro decisões imediatas precisam ser confirmadas: vocabulário de readiness, política de override, conteúdo mínimo do Assurance Core e versionamento ao reabrir etapas.

2. Objetivo do trabalho e deste documento

2.1 Objetivo do trabalho de definição

O trabalho ProductMind tem como objetivo transformar a concepção do SystemBuilder em uma **definição funcional e experiencial explícita, consistente e verificável**, capaz de orientar etapas posteriores de design detalhado, engenharia, testes e preparação de produto.

O escopo atual é **Product Design / definição do produto**. Portanto, esta evolução não deve ser interpretada como conclusão de:

- Product Discovery ou validação de problema/mercado;
- validação do modelo comercial;
- arquitetura de engenharia final;
- implementação ou prova de conceito;

- validação completa de segurança, privacidade ou compliance;
- revisão jurídica de licenciamento, distribuição ou marca;
- homologação definitiva de integrações com provedores de IA.

2.2 Objetivo deste relatório

Este relatório reorganiza o `ESTADO_SystemBuilder_v0.6.md` para permitir que pessoas que **não participaram do processo até aqui** compreendam:

1. o que é o SystemBuilder e qual problema pretende resolver;
2. como o conceito evoluiu;
3. quais decisões já foram tomadas e por quê;
4. qual é a definição atual do produto;
5. quais riscos, pendências e dependências permanecem;
6. quais decisões são imediatamente necessárias;
7. qual é o caminho provável até uma definição suficientemente madura para o PDD final.

O documento é uma **visão de comunicação e progresso**, não substitui o ESTADO canônico como fonte operacional de verdade.

2.3 Convenção epistemológica

Ao longo do processo, as informações são classificadas para separar decisão de fato, inferência e hipótese:

- **USR:** informação ou decisão confirmada pelo usuário;
- **FAT:** fato externo sustentado por evidência;
- **INF:** inferência técnica razoável;
- **HIP:** hipótese de trabalho ainda não confirmada;
- **REC:** recomendação do Product Designer ou especialista;
- **PEND:** questão ainda aberta;
- **DESC:** item deliberadamente descartado.

No relatório, a narrativa principal privilegia o **estado vigente**. Os registros completos de rastreabilidade são preservados nos anexos.

3. Descrição atual do produto

3.1 O que é o SystemBuilder

O SystemBuilder é uma **plataforma de orquestração guiada para desenvolvimento de software com IA**. Seu papel não é apenas oferecer acesso a modelos ou agentes, mas organizar a transformação de uma ideia em software implantado por meio de um processo estruturado, progressivo, verificável e adaptável ao perfil, ao objetivo e ao risco do projeto.

O usuário deverá poder iniciar um projeto do zero ou entrar em um ponto intermediário do processo. O SB identifica o contexto disponível, verifica o que falta, solicita somente as informações ou decisões necessárias e apresenta ações compatíveis com a etapa corrente, o modo de interação e a política de autonomia escolhida.

A experiência deverá esconder a complexidade operacional do BMAD sem ocultar sua utilização. O usuário interage com o **SystemBuilder**, não com uma coleção de prompts ou workflows BMAD expostos diretamente.

3.2 Proposta de valor

A proposta de valor combina dois eixos.

Facilitação e produtividade

- reduzir a curva de aprendizado do desenvolvimento com IA;
- reduzir setup e atrito entre ferramentas, artefatos e etapas;
- conduzir o usuário pelo fluxo, sem exigir domínio prévio de todo o ciclo de engenharia;
- solicitar decisões humanas apenas quando materialmente necessárias;
- permitir diferentes pontos de entrada e retomada de projetos.

Production assurance e continuidade

- tornar ações relevantes rastreáveis;
- preservar proveniência de decisões e artefatos;
- relacionar execução, estado, permissões e evidências;
- suportar gates humanos para operações de maior risco;
- identificar risco residual;
- favorecer entendimento, verificação, operação, migração e evolução posterior do software;
- reduzir falhas típicas de processos baseados em IA, como alucinação, perda de contexto, prompt injection, permissões excessivas, specification gaming e falta de governança.

3.3 Ciclo de desenvolvimento suportado

A primeira versão utilizável deverá cobrir o ciclo completo:

1. **Ideia vaga → Brainstorm**
2. **Ideia avançada → Validação**
3. **Ideia testada → Pesquisa e decisão de marketing**
4. **Conceito estabelecido → Product Brief / PRFAQ**
5. **PRD → Design UX**
6. **Design and Experience → Design Architecture**
7. **Architecture → Planning**
8. **Epics and Stories → Coding**
9. **Sistema pronto → Test / Review / Deploy**
10. **Sistema operando**

A profundidade não precisa ser uniforme em todas as etapas na primeira versão, mas o percurso ponta a ponta deve existir.

3.4 Ciclo de Coding por story

A fase de Coding foi concebida como um ciclo repetível por story:

1. story + requisitos → `create-story`;
2. story escrita → revisão adversarial;
3. story revisada → security review;
4. story finalizada → Code Writing;
5. código → testing;
6. código testado → Code Review + Correction;
7. código revisado → Security Review + Corrections;
8. story concluída;
9. ao final do epic → retrospective e registro de lições aprendidas.

Esse desenho ainda será especializado em contratos formais de entrada, saída, readiness, evidência e critérios de fechamento.

3.5 Jornada principal provisória

A jornada operacional consolidada até aqui é:

1. autenticar;
2. conectar a IA do usuário;
3. escolher entre continuar um projeto existente ou iniciar um novo;
4. em um novo projeto, selecionar o ponto inicial do processo;
5. fornecer ou importar os artefatos exigidos;
6. permitir que o SB verifique readiness/prontidão;
7. receber orientação contextual para a etapa atual;
8. fornecer decisões ou informações humanas quando necessário;
9. executar ou liberar ações compatíveis com etapa, modo de interação e política de autonomia;
10. registrar artefatos, decisões, evidências e transições de estado;
11. transicionar para a etapa seguinte quando os critérios aplicáveis forem satisfeitos;

12. repetir o ciclo até deploy/sistema operando.

Essa jornada permanece provisória quanto ao detalhamento de exceções e microinterações, mas é consistente com as decisões estruturais vigentes.

3.6 Escopo futuro, fora da primeira fase

Estão explicitamente reservados para evolução posterior:

- operação, manutenção e evolução contínua pós-deploy;
- promoção, venda e pós-venda;
- planos em que a IA seja fornecida ou revendida pelo próprio SB;
- colaboração multiusuário e workspaces;
- paralelismo amplo de runs/agentes.

4. Usuários e modelo de uso

4.1 Segmentação operacional

A segmentação adotada não depende apenas de rótulos profissionais. Ela considera responsabilidade sobre o código, criticidade do software, presença de usuários/dados reais, estrutura de equipe e governança exigida.

Faixa	Perfil operacional	Situação predominante
1	Usuário ocasional / não técnico	Quer obter algo útil sem montar um ambiente complexo de desenvolvimento
2	Maker / indie	Constrói e publica repetidamente, geralmente sozinho ou com equipe mínima
3	Founder / PM / designer	Quer transformar hipótese, produto ou feature em software utilizável
4	Desenvolvedor profissional	Busca produtividade sem perder contexto, controle e capacidade de revisão
5	Equipe de engenharia	Requer colaboração, coordenação, ambientes e consistência multiusuário
6	Enterprise / platform / security	Requer identidade, políticas, auditoria, compliance e governança em escala

O **envelope funcional inicial** cobre as faixas **1 a 4**. As faixas 5 e 6 não são alvo prioritário da primeira fase. O beachhead comercial poderá ser mais estreito sem reduzir esse envelope funcional.

4.2 Modo de interação e política de autonomia

Uma decisão importante foi separar duas dimensões que inicialmente poderiam ser confundidas.

Dimensão	Valores iniciais	O que controla
Modo de interação	Guiado / Avançado	Quanto da complexidade, opções e controles o usuário vê
Política de autonomia	Conservadora / Supervisionada / Autônoma	Quanto o SB pode decidir/executar sem aprovação humana

O default conceitual será **Guiado + Supervisionada**.

Essa separação permite, por exemplo:

- usuário não técnico em projeto crítico: **Guiado + Conservadora**;
- founder buscando velocidade de experimentação: **Guiado + Autônoma**;

- engenheiro sênior em sistema crítico: **Avançado + Conservadora**;
- especialista conduzindo protótipo: **Avançado + Autônoma**.

Configurações mais granulares por categoria de ação poderão ser expostas progressivamente, sobretudo no modo Avançado.

4.3 Modelo de colaboração do MVP

Cada projeto terá **um único usuário principal** na primeira versão utilizável. Colaboração multiusuário, times e workspaces ficam para evolução posterior.

Apesar dessa simplificação, os conceitos de usuário, projeto, ownership e execução devem permanecer separados para que a evolução futura não exija reconstrução estrutural.

5. Arquitetura funcional do produto

5.1 Relação entre SystemBuilder e BMAD

A decisão adotada é **native-first**:

- o BMAD permanece autoridade sobre o conteúdo e os estados semânticos que já governa adequadamente;
- o SB não cria uma cópia semântica concorrente desses estados;
- o SB mantém apenas a semântica transversal própria do produto.

Em termos conceituais:

```
SYSTEMBUILDER
  UX / ORQUESTRAÇÃO / ASSURANCE
                |
         SB CONTROL PLANE
                |
       ADAPTER BMAD VERSIONADO
                |
      BMAD HOMOLOGADO / NATIVO
```

5.2 Responsabilidade do SB Control Plane

O SB deverá ser autoridade sobre, pelo menos:

- identidade de projeto e de execução;
- `run_id`;
- etapa macro do SB;
- modo de interação;
- política de autonomia;
- permissões e gates;
- aprovações humanas;
- proveniência;
- índice de evidências;
- risco residual;
- relações entre artefatos e runs;
- staleness cross-workflow;
- política de homologação do BMAD;
- reconciliação necessária para projetar um estado consolidado ao usuário.

5.3 Responsabilidade nativa do BMAD

Quando adequada, a representação nativa do BMAD continua sendo a autoridade de conteúdo e estado que o próprio workflow governa.

A análise especializada identificou, no BMad Method v6.12.0, mecanismos como:

- memlogs canônicos em determinados workflows;
- artefatos derivados;

- frontmatter;
- `sprint-status.yaml`;
- `stories.yaml`;
- readiness gates;
- snapshots / render manifests;
- tracking de implementação.

Esses mecanismos são relevantes, mas distribuídos e não formam, sozinhos, o ledger transversal de assurance necessário ao SB.

5.4 Adapter versionado / anti-corruption layer

O restante do produto não deverá depender diretamente de paths, nomes de arquivos, enums ou skills internos do BMAD.

Cada versão homologada deverá ser mediada por um adaptador capaz, conceitualmente, de executar operações equivalentes a:

```
inspect
  -> validate_entry
  -> invoke
  -> collect
  -> reconcile
  -> project_state
```

Isso reduz acoplamento e permite que novas versões do BMAD sejam avaliadas, testadas e homologadas antes de sua incorporação.

5.5 Política de atualização do BMAD

O SB não seguirá automaticamente o upstream.

Uma nova versão candidata deverá passar por:

1. identificação de versão/tag/commit e módulos aplicáveis;
2. análise de mudanças de contrato e comportamento;
3. testes de compatibilidade;
4. avaliação de impacto sobre customizações;
5. ajustes no adapter, quando necessários;
6. homologação explícita;
7. migração controlada.

6. Assurance, segurança e governança da autonomia

6.1 Assurance como parte do produto

Assurance não será uma opção binária. A decisão atual é:

Assurance Core obrigatório + profundidade graduada conforme etapa, risco, criticidade e contexto.

O Assurance Core começa já em **Ideação/Brainstorm**, mas com profundidade leve. O rigor aumenta conforme o produto passa a gerar artefatos formais, código, mudanças de estado, integrações e operações com maior impacto.

6.2 Direções já definidas

A camada de assurance deverá suportar, conforme aplicabilidade e profundidade:

- requisitos recuperados e explicitados;
- arquitetura e modelo de dados explícitos;
- baseline reproduzível de build/test/lint/security;
- tratamento de riscos de schema e estado;
- remediações pequenas e rastreáveis;

- gates humanos para operações de alto risco;
- dossier de handoff;
- rastreabilidade de custo e decisões;
- evidência da versão/configuração do engine utilizado;
- proveniência e ownership dos artefatos;
- identificação de verificações e risco residual.

6.3 Classes de falha e risco consideradas

O produto deverá considerar, sem pressupor uma única resposta técnica para todas elas:

- vieses e discriminação;
- alucinações e confabulações;
- falta de explicabilidade e rastreabilidade;
- degradação de desempenho / data drift, quando aplicável;
- ataques adversários e vulnerabilidades;
- automação de processos ineficientes;
- ancoragem e complacência humana;
- desalinhamento com objetivos de negócio;
- ausência de governança e monitoramento;
- problemas de integração e infraestrutura;
- specification gaming / reward hacking;
- comportamentos emergentes ocultos;
- deceptive alignment;
- prompt injection;
- permissões excessivas e falta de isolamento de escopo;
- data poisoning indireto;
- riscos de cadeia de suprimentos de IA.

6.4 Segurança, privacidade e compliance ainda abertos

Ainda precisam ser definidos:

- tratamento de código-fonte, prompts e dados de projeto;
- modelo de armazenamento e proteção de credenciais/tokens;
- tratamento de propriedade intelectual;
- política de autorização por tipo de ação;
- modelo de logs e retenção;
- necessidades específicas de privacidade e compliance conforme mercado;
- revisão jurídica de licença, distribuição e trademark do BMAD.

7. State Model atual

7.1 Estado macro e estado local

O SB terá dois níveis complementares:

Estado macro do projeto: representa a etapa primária do ciclo em que o projeto se encontra.

Estado local da etapa: representa o progresso e as condições internas daquela etapa.

No MVP, haverá **uma única etapa macro primária ativa por vez**. Revisões e subfluxos auxiliares podem existir sem transformar o projeto em múltiplas etapas macro simultâneas.

7.2 Vocabulário local baseline

O vocabulário confirmado é:

Estado	Significado operacional
`NOT_STARTED`	etapa ainda não iniciada
`READINESS`	verificação das condições de entrada

Estado	Significado operacional
`ACTIVE`	execução normal da etapa
`AWAITING_USER`	depende de informação, decisão ou aprovação do usuário
`BLOCKED`	existe condição indispensável que impede prosseguir
`READY_TO_CLOSE`	execução principal concluída, aguardando validação de fechamento
`COMPLETED`	contrato de saída integralmente satisfeito
`STALE`	confiabilidade potencialmente afetada por mudança upstream
`REOPENED`	etapa anteriormente concluída foi reativada deliberadamente

7.3 Contrato de saída como condição de conclusão

Uma etapa **não é concluída apenas porque uma IA ou workflow terminou sua execução**.

`COMPLETED` exige satisfação do respectivo contrato de saída, incluindo, conforme a etapa:

- outputs obrigatórios;
- verificações;
- gates;
- decisões humanas;
- evidências;
- tratamento de bloqueadores;
- critérios de aceitação aplicáveis.

7.4 Mudanças upstream e staleness

Alterar uma decisão ou artefato upstream não deve apagar automaticamente o trabalho downstream.

O comportamento adotado é:

```
mudança upstream
      |
análise de impacto
      |
+----------------------------+
| sem impacto material       | -> mantém validade
| impacto possível           | -> REQUIRES_REVIEW
| evidência invalidada       | -> STALE
+----------------------------+
```

A granularidade exata das regras de impacto ainda precisa ser definida por relação entre etapas e artefatos.

7.5 Entrada intermediária: IMPORT_ADOPTION

Quando um projeto entra no SB em uma etapa intermediária, não deverá simplesmente “começar daquela etapa”.

O fluxo será controlado:

```
seleção da etapa inicial
      |
IMPORT_ADOPTION
      |
identificação dos artefatos existentes
      |
registro de origem / ownership / proveniência
      |
validação de completude e consistência
      |
```

```
readiness da etapa escolhida
      |
correção das lacunas, se necessárias
      |
entrada efetiva na etapa
```

O objetivo é permitir flexibilidade sem sacrificar rastreabilidade e confiabilidade.

8. Capacidades e requisitos atuais

8.1 Capacidades funcionais principais

O SB deverá:

- criar e retomar projetos;
- permitir seleção da etapa inicial;
- validar pré-requisitos;
- solicitar informações ausentes;
- orquestrar o fluxo por etapas;
- apresentar ações compatíveis com etapa, modo e autonomia;
- conectar e utilizar IA do usuário;
- controlar a versão BMAD homologada;
- executar o ciclo de criação/revisão/teste/segurança de stories;
- registrar retrospectiva por epic;
- registrar decisões e evidências relevantes;
- tornar identificáveis estado e permissões associados a ações;
- suportar human gates;
- produzir handoffs que favoreçam continuidade;
- correlacionar runs a engine, inputs e outputs;
- identificar ownership dos artefatos;
- detectar inconsistências de estado;
- testar contratos por versão BMAD;
- tratar staleness/invalidation;
- preservar preparação para futuro paralelismo;
- manter Assurance Core;
- graduar assurance por risco;
- suportar um usuário principal por projeto;
- manter uma única etapa macro primária ativa;
- suportar reabertura;
- realizar análise de impacto;
- executar `IMPORT_ADOPTION`;
- aplicar o vocabulário baseline de estados locais.

8.2 Requisitos não funcionais já estabelecidos

- conexão com IA deve minimizar atrito;
- a interface é própria do SB e deve ser amigável;
- updates do BMAD não podem ser automáticos;
- a estrutura inicial não pode impedir futura flexibilização de autonomia, intervenção, concorrência ou colaboração;
- autonomia não deve estar rigidamente acoplada a personas.

8.3 Dados e integrações ainda em definição

Permanecem abertas:

- shortlist de provedores de IA;
- viabilidade real de conexão por conta/assinatura;
- conexão via API/credencial;

- persistência e versionamento físico de artefatos;
- integração com repositórios;
- IDEs;
- CI/CD;
- hosting/deploy;
- ferramentas de gestão.

9. Histórico de evolução e decisões

9.1 Evolução por versão do ESTADO

Versão	Evolução principal	Consequência
0.1	Baseline inicial da concepção	Criou a primeira fonte canônica
0.2	Segmentação, modos, autonomia supervisionada, ciclo completo e assurance como diferencial	O produto deixa de ser apenas uma camada de facilitação
0.3	Reconciliação do especialista BMAD	Evidencia mecanismos nativos e gaps transversais
0.4	Native-first, adapter versionado, serialização do MVP, Assurance Core, autonomia independente do modo	Fecha os principais princípios de arquitetura funcional
0.5	Envelope 1–4, assurance desde Ideação e usuário único por projeto	Fecha segmentação funcional e colaboração inicial
0.6	State Model macro, Exit Contracts, staleness, `IMPORT_ADOPTION` e estados locais	Habilita a definição formal dos contratos de etapa

9.2 Decisões confirmadas

Todas as decisões a seguir foram registradas com autoridade do usuário em 23/09/2026.

ID	Decisão vigente	Razão / impacto principal
D-001	Nome provisório: SystemBuilder / SB	Identificação do produto
D-002	Primeira versão utilizável cobre ideia → deploy	Define amplitude do produto inicial
D-003	Fluxo principalmente baseado no BMAD customizado	Define dependência metodológica central
D-004	BMAD será versionado de forma controlada	Evita atualização upstream automática
D-005	IA inicial será conectada/fornecida pelo usuário	Define modelo inicial de consumo de IA
D-006	Interface oculta a complexidade operacional do BMAD	SB é a experiência principal
D-007	Modos Guiado e Avançado	Cria dois níveis iniciais de exposição de controle
D-008	Autonomia Supervisionada é o default	Equilibra velocidade e controle
D-009	Autonomia pode variar por objetivo, risco e criticidade	Evita persona fixa como proxy de risco
D-010	Assurance, evidência e rastreabilidade são diferenciais centrais	Reposiciona o valor do produto
D-011	Segmentos 5–6 fora do alvo prioritário inicial	Reduz complexidade de governança no MVP
D-012	Reutilizar gestão nativa BMAD antes de criar estado redundante	Evita duplicação

ID	Decisão vigente	Razão / impacto principal
D-013	Pós-deploy ampliado e comercial ficam para fases futuras	Controla escopo
D-014	Native-first: BMAD native state + SB control plane	Define ownership funcional de estado
D-015	Adapter BMAD versionado / anti-corruption layer	Desacopla o produto do framework
D-016	Execução serial em escopos de conflito, preparada para paralelo	Reduz risco do MVP sem engessar futuro
D-017	Assurance Core obrigatório e graduado	Preserva diferencial desde o MVP
D-018	Modo e autonomia são dimensões independentes; default Guiado + Supervisionada	Melhora coerência comportamental
D-019	Envelope funcional 1–4; beachhead pode ser menor	Separa produto de go-to-market
D-020	Assurance começa em Ideação e aumenta em profundidade	Cria cadeia de proveniência desde a origem
D-021	Um usuário principal por projeto no MVP	Simplifica colaboração inicial
D-022	Uma etapa macro primária ativa por vez	Simplifica orquestração e UX
D-023	`COMPLETED` depende de Exit Contract	Evita conclusão falsa por simples término de run
D-024	Mudança upstream gera impacto/staleness, não destruição automática	Preserva trabalho e rastreabilidade
D-025	Entrada intermediária passa por `IMPORT_ADOPTION`	Controla ingestão de trabalho externo
D-026	Vocabulário local baseline confirmado	Padroniza o State Model

10. Parecer especializado BMAD e impacto

10.1 Objetivo do handoff

O handoff `HO-SB-BMAD-001`, baseado no `ESTADO_SystemBuilder_v0.2.md`, avaliou em que profundidade o BMAD já oferecia gestão de estado, artefatos, contratos de entrada/saída e handoffs e quais lacunas justificariam uma camada própria do SB.

10.2 Conclusões incorporadas

A análise, ancorada no BMad Method v6.12.0, encontrou:

- mecanismos substanciais de persistência, retomada, artefatos canônicos e tracking;
- contratos fortes em algumas áreas e interfaces file-based/comportamentais em outras;
- ausência de um ledger transversal único com a semântica de assurance do SB;
- gaps conhecidos de consistência/atribuição em cenários específicos;
- vantagem em preservar ownership e fontes nativas;
- necessidade de reconciliação explícita quando fontes nativas divergirem;
- benefício de um adapter versionado e contract tests por versão homologada.

Esses achados fundamentaram diretamente D-014 e D-015 e os requisitos RF-015 a RF-019.

11. Pendências atuais e questões abertas

A leitura do estado deve distinguir pendências **efetivamente abertas** de registros editoriais antigos que já foram superados por decisões posteriores.

11.1 Pendências efetivamente abertas

Tema	Situação atual	Momento recomendado
Beachhead comercial dentro do envelope 1–4	Ainda não escolhido	Discovery / go-to-market
Licença, distribuição e trademark BMAD	Revisão jurídica específica ainda necessária	Antes de comercialização/arquitetura final de distribuição
Provedores de IA e formas de conexão	Shortlist e viabilidade ainda abertas	Após fechar requisitos de integração
Regras detalhadas de reabertura e invalidação	Princípio definido; granularidade ainda aberta	Durante Stage Contracts e dependency model
Métricas e eventos	Candidatas existem; métricas finais não definidas	Após jornadas críticas
Integrações repo/IDE/CI/CD/deploy	Ainda abertas	Após macroarquitetura funcional
Conteúdo mínimo do Assurance Core	Decisão estrutural tomada; campos mínimos pendentes	Decisão imediata
Inventário BMAD skill-by-skill	Ainda não realizado	Depois de estabilizar o fluxo v1
Persistência física/versionamento de artefatos	Ainda aberta	Preparação para engenharia
Segurança/privacidade de código, prompts e credenciais	Ainda aberta	Antes de engenharia detalhada
Permissões internas do usuário principal	Ainda abertas	Ao detalhar autonomy/gates

11.2 Nota de integridade do ESTADO v0.6

Durante a preparação deste relatório foram identificados **registros duplicados com redação defasada dentro do próprio v0.6**. Para fins deste relatório, foi aplicada a regra de precedência pela decisão mais recente e explícita, sem alterar silenciosamente o arquivo canônico.

As inconsistências editoriais identificadas são:

1. a seção “Princípio comportamental” ainda apresenta a separação modo/autonomia como REC, mas D-018 e E-023 já a confirmam como USR;
2. P-001 ainda aparece como pendente, embora D-019 tenha confirmado o envelope funcional 1–4;
3. P-002 ainda aparece como pendente, embora D-018 tenha confirmado a separação modo/autonomia;
4. P-003 ainda aparece como pendente, embora D-020 tenha confirmado assurance desde Ideação;
5. P-004 ainda aparece como pendente, embora D-021 tenha confirmado um usuário principal por projeto;
6. P-005 e o registro do handoff ainda mencionam decisão arquitetural pendente, mas D-014/D-015 já confirmaram a fronteira native-first e o adapter;
7. alguns tratamentos de risco mantêm linguagem condicional anterior às decisões já confirmadas.

Interpretação aplicada neste relatório: decisões D-014 a D-026 e registros epistemológicos E-018 a E-032 prevalecem sobre formulações anteriores repetidas que não foram editorialmente atualizadas.

Recomenda-se, como ação de housekeeping, corrigir essas duplicidades em um futuro `ESTADO_SystemBuilder_v0.6.1` ou na próxima versão material do ESTADO, sem alterar significado de produto.

12. Riscos, restrições e dependências

12.1 Riscos registrados

ID	Risco	Impacto	Tratamento vigente ou planejado
R-001	Dependência relevante do BMAD	Alto	Versionamento, customizações controladas, licença e homologação

ID	Risco	Impacto	Tratamento vigente ou planejado
R-002	Duplicação do estado BMAD	Alto	Native-first; SB mantém apenas semântica transversal
R-003	Complexidade de múltiplos provedores de IA	Alto	Abstração + shortlist e pesquisa técnica
R-004	Amplitude excessiva do ciclo ideia→deploy	Alto	Contratos mínimos e profundidade desigual por etapa
R-005	Confundir qualificação técnica com autonomia	Alto	D-018 já separa as dimensões
R-006	UX/arquitetura engessada	Alto	Configuração por modo/política e adapter
R-007	Importação intermediária inconsistente	Alto	`IMPORT_ADOPTION` e Entry Contracts
R-008	Credenciais, código e propriedade intelectual	Alto	Modelo de segurança/privacidade ainda necessário
R-009	Assurance virar apenas documentação	Alto	Associar evidência a gates, estado e transições
R-010	Divergência entre ledgers BMAD	Alto	Reconciliação antes de transições críticas
R-011	Upgrade BMAD alterar semântica de contratos	Alto	Adapter + contract tests + homologação
R-012	Prompt ser confundido com enforcement de autorização	Alto	Enforcement no SB control plane
R-013	Perda de ownership/proveniência	Alto	Preservar owner, path, hash e origem
R-014	Single-user arquiteturalmente irreversível	Médio/Alto	Separar conceitos de User/Project/Ownership
R-015	Cascata excessiva de staleness	Médio/Alto	Regras de impacto proporcionais
R-016	Exit Contracts burocratizarem fases iniciais	Médio	Critérios proporcionais a etapa/risco
R-017	`IMPORT_ADOPTION` gerar atrito excessivo	Médio	Coleta incremental e validação automatizada

12.2 Restrições atuais

- primeira fase limitada ao ciclo até sistema operando após deploy;
- IA fornecida pelo usuário;
- um usuário principal por projeto;
- concorrência ampla fora do MVP;
- BMAD homologado e versionado, sem updates automáticos;
- colaboração/workspaces e frentes comerciais pós-deploy ficam para evolução posterior.

13. Métricas e critérios de sucesso ainda em construção

As métricas finais ainda não foram escolhidas. O ESTADO registra como candidatas:

- tempo para iniciar um projeto;
- taxa de conclusão de etapas;
- número de intervenções obrigatórias do usuário;
- bloqueios por falta de informação;

- retrabalho/reabertura;
- sucesso na conexão com IA;
- abandono por etapa;
- percentual de ações relevantes com rastreabilidade completa;
- cumprimento de gates críticos;
- falhas detectadas antes da transição;
- completude de dossier/handoff.

Essas métricas deverão ser refinadas após as jornadas e os contratos críticos estarem mais maduros.

14. Próximo passo imediato — decisões pendentes

A seção abaixo preserva o conteúdo da rodada imediatamente anterior, reorganizado apenas visualmente para integração neste relatório.

14.1 Objeto seguinte: contrato genérico de uma etapa

A recomendação é não começar detalhando Brainstorm, PRD, Architecture, Coding etc. individualmente. Primeiro deve ser definido um **Stage Contract genérico**, que todas as etapas especializarão.

Conceitualmente:

```
STAGE CONTRACT

IDENTIDADE
  etapa / versão / engine-capability

ENTRADA
  preconditions
  required inputs
  optional inputs
  provenance

READINESS
  validations
  gaps
  warnings
  blockers
  overrides permitidos

EXECUÇÃO
  actions
  runs
  autonomy policy
  permissions / gates
  human decisions

SAÍDA
  outputs
  evidence
  findings
  residual risk

FECHAMENTO
  exit criteria
  required approvals
  completion status
```

```
DEPENDÊNCIAS
  downstream impacts
  staleness rules
  reopening rules
```

Esse contrato será uma abstração do **SB**. Quando BMAD já possuir mecanismos equivalentes, o adapter os utilizará em vez de duplicá-los.

14.2 Decisão 1 — resultado do Readiness

Recomenda-se padronizar quatro resultados no SB:

- `PASS`: permite prosseguir;
- `CONCERNS`: permite prosseguir mediante política/gate apropriado;
- `FAIL`: critérios não foram atingidos e exige correção/reavaliação;
- `BLOCKED`: falta condição indispensável e a etapa não pode prosseguir.

A distinção entre `FAIL` e `BLOCKED` é útil: um documento com qualidade insuficiente pode `FAIL`; ausência de uma autorização indispensável pode `BLOCK`.

Recomendação: adotar `PASS / CONCERNS / FAIL / BLOCKED`.

14.3 Decisão 2 — Override

Recomenda-se que:

- `CONCERNS` possa admitir override explícito e rastreável pelo usuário principal, respeitando a política de autonomia;
- `FAIL` só admita override quando o contrato da etapa declarar expressamente essa possibilidade e registrar risco residual;
- `BLOCKED` não possua bypass genérico: é necessário remover a condição bloqueadora ou seguir uma exceção formal específica previamente prevista.

Isso impede que “continuar mesmo assim” se transforme em uma porta universal para contornar assurance.

Recomendação: adotar a política acima.

14.4 Decisão 3 — Assurance Core mínimo

Recomenda-se que toda execução relevante, mesmo em Brainstorm, registre pelo menos:

```
project_id
stage
run_id
timestamp
actor/origin
engine + version
inputs/references
outputs/references
decisões humanas relevantes
gates aplicáveis
resultado
evidence refs
findings relevantes
residual risk, quando houver
```

A profundidade cresce ao longo do ciclo, mas essa identidade mínima cria a cadeia de proveniência desde a origem da ideia.

Recomendação: adotar esse conjunto como baseline mínima do Assurance Core, sujeito ao detalhamento de schema nas etapas posteriores.

14.5 Decisão 4 — versionamento ao reabrir etapas

Recomenda-se que reabrir uma etapa `COMPLETED` **não sobrescreva silenciosamente sua baseline anterior**.

A conclusão anterior permanece identificável e a etapa entra em `REOPENED`, produzindo uma nova revisão:

```
PRD v1 - COMPLETED
        |
Architecture v1
        |
mudança de produto
        |
PRD v1 - preservado
PRD v2 - REOPENED -> ACTIVE -> COMPLETED
        |
Impact Analysis
        |
Architecture v1 - STALE
```

Esse comportamento é especialmente importante para o diferencial de rastreabilidade do SB.

Recomendação: preservar as baselines concluídas e gerar nova revisão ao reabrir.

14.6 Recomendação consolidada da rodada

A recomendação é **adotar as quatro propostas**. Elas formam um conjunto coerente:

- readiness determina se a etapa pode executar/avançar;
- override governa exceções;
- Assurance Core prova o que aconteceu;
- versionamento preserva a história quando decisões mudam.

Se confirmadas, o próximo artefato deverá ser o **Stage Contract v0.1**, aplicado inicialmente a três etapas deliberadamente diferentes:

1. **Ideação / Brainstorm**;
2. **PRD**;
3. **Coding / Story**.

A finalidade é testar se a abstração funciona em trabalho conceitual, documental e de engenharia antes de replicá-la para o ciclo completo.

15. Próximos passos prováveis até a definição final

Os passos abaixo são uma projeção de trabalho, não decisões fechadas de produto. A sequência poderá ser ajustada conforme os achados de cada rodada.

15.1 Fechar as quatro decisões imediatas

Confirmar ou ajustar:

1. vocabulário de Readiness;
2. política de override;
3. baseline do Assurance Core;
4. regra de versionamento/reabertura.

15.2 Produzir o Stage Contract v0.1

Formalizar o contrato genérico contendo:

- identidade da etapa;
- Entry Contract;
- inputs obrigatórios e opcionais;
- proveniência;

- readiness;
- ações e runs;
- autonomia;
- gates/permissões;
- decisões humanas;
- outputs;
- evidências;
- findings e risco residual;
- Exit Contract;
- relações downstream;
- staleness e reopening.

15.3 Aplicar o contrato a três etapas representativas

Modelar:

- Ideação/Brainstorm;
- PRD;
- Coding/Story.

O objetivo é encontrar lacunas do contrato genérico antes de replicá-lo.

15.4 Revisão adversarial intermediária

Submeter os três contratos representativos a revisão independente, com foco em:

- contradições;
- exceções não tratadas;
- burocratização desnecessária;
- caminhos de bypass;
- staleness excessivo;
- rastreabilidade insuficiente;
- inconsistência BMAD ↔ SB;
- impacto de diferentes políticas de autonomia.

15.5 Reconciliar e generalizar o modelo

Após revisão:

- corrigir o Stage Contract;
- fechar a semântica de estados;
- fechar regras de transição;
- definir contratos das demais etapas ideia→deploy;
- detalhar relações de dependência/invalidação.

15.6 Definir modelo de dados funcional e artefatos

Sem antecipar arquitetura de engenharia, especificar:

- entidades conceituais: Project, Stage, Run, ArtifactRef, Gate, Decision, Evidence, Finding etc.;
- ownership;
- versão/revisão;
- proveniência;
- relações e dependências;
- requisitos de persistência e recuperação.

15.7 Detalhar integração com IA e BMAD

- selecionar provedores/runtimes prioritários;
- pesquisar viabilidade de conexão por conta/assinatura e API;
- inventariar skills BMAD realmente utilizadas no fluxo v1;
- mapear contratos por skill;

- detalhar responsabilidades do adapter;
- preparar critérios de homologação e contract tests.

15.8 Segurança, privacidade, compliance e licenciamento

- definir tratamento de credenciais e secrets;
- definir isolamento e autorização;
- definir retenção e exposição de evidências;
- definir tratamento de código e propriedade intelectual;
- executar análise específica de licença/trademark/distribuição do BMAD;
- identificar necessidades regulatórias conforme mercados efetivamente escolhidos.

15.9 UX e jornadas detalhadas

- dashboard e retomada;
- criação de projeto;
- conexão de IA;
- seleção do ponto inicial;
- `IMPORT_ADOPTION`;
- execução de etapa;
- `AWAITING_USER`;
- gates/override;
- staleness/revisão;
- reabertura;
- finalização/deploy;
- modos Guiado e Avançado.

Wireframes e protótipos só serão necessários quando trouxerem ganho material para decisões.

15.10 Métricas, eventos e critérios de aceitação

Transformar métricas candidatas em indicadores selecionados e definir:

- eventos relevantes;
- critérios de aceitação por capacidade crítica;
- evidências exigidas por Exit Contract;
- indicadores de assurance/completude;
- limites de desempenho/experiência aplicáveis.

15.11 Gate de maturidade e revisão adversarial final

Antes de concluir o Product Definition Document:

- verificar ausência de bloqueadores;
- avaliar problema, usuários, valor, escopo, funcionalidades, fluxos, regras e critérios de aceitação;
- registrar ressalvas aceitas;
- executar revisão adversarial final;
- reconciliar todos os achados materiais.

15.12 PDD final

Quando o gate estiver satisfeito, produzir o **PDD_SystemBuilder_vN.N.md**, declarando claramente:

- o que foi definido;
- o que permanece aberto;
- o que não foi validado em Discovery;
- quais estudos e decisões ficam para engenharia, segurança, jurídico ou mercado.

16. Limites atuais da definição

O produto já possui uma arquitetura funcional conceitual relevante, mas **ainda não está pronto para ser tratado como especificação de engenharia**.

Em particular, não foram concluídos:

- Product Discovery;
- validação de mercado e precificação;
- prova de conceito de integração;
- arquitetura técnica de produção;
- modelo físico de persistência;
- shortlist/homologação de IAs;
- segurança completa;
- privacidade/compliance;
- revisão jurídica;
- contratos detalhados de todas as etapas;
- critérios de aceitação completos;
- métricas finais;
- revisão adversarial final.

Essa distinção é deliberada: o objetivo atual é reduzir ambiguidade de produto sem criar falsa certeza sobre disciplinas ainda não executadas.

Até o ESTADO v0.6, **não há itens formalmente registrados como descartados (`DESC`)**.

Anexo A — Registro completo de requisitos funcionais

ID	Requisito
RF-001	Criar e retomar projetos
RF-002	Permitir escolha da etapa inicial
RF-003	Validar pré-requisitos da etapa inicial
RF-004	Solicitar informações ausentes
RF-005	Orquestrar o fluxo por etapas
RF-006	Expor ações compatíveis com etapa, modo e política de autonomia
RF-007	Conectar e utilizar IA do usuário
RF-008	Manter versão controlada do BMAD no ambiente do SB
RF-009	Aplicar ciclo de criação/revisão/teste/segurança por story durante Coding
RF-010	Registrar retrospectiva e lições aprendidas ao final de cada epic
RF-011	Registrar decisões e evidências relevantes de execução
RF-012	Tornar identificáveis permissões e estado associados a ações relevantes
RF-013	Suportar gates humanos em operações de maior risco
RF-014	Produzir artefatos/handoffs que favoreçam compreensão, continuidade e evolução do software
RF-015	Correlacionar execução a projeto, run, versão/tag/commit BMAD, configuração/snapshot, inputs e outputs
RF-016	Identificar autoridade de cada artefato relevante: BMAD, SB ou externo
RF-017	Detectar inconsistências materiais entre representações nativas usadas para autorizar uma transição e bloquear decisão automática sem reconciliação segura
RF-018	Manter mapa de capacidades/contratos testado por versão BMAD homologada
RF-019	Explicitar staleness/invalidation quando mudança de input exigir reverificação downstream
RF-020	Operar serialmente em escopos de conflito no MVP, preservando evolução futura para paralelo

ID	Requisito
RF-021	Registrar núcleo obrigatório de assurance por execução relevante
RF-022	Suportar profundidade graduada de assurance
RF-023	Ativar Assurance Core desde Ideação/Brainstorm
RF-024	Suportar um único usuário principal por projeto no MVP
RF-025	Não depender de colaboração multiusuário/workspaces para completar ideia→deploy
RF-026	Manter uma única etapa macro primária ativa por vez
RF-027	Exigir Exit Contract verificável para `COMPLETED`
RF-028	Suportar reabertura deliberada de etapas concluídas
RF-029	Executar análise de impacto após mudanças upstream relevantes
RF-030	Marcar itens como `STALE` ou `REQUIRES_REVIEW` em vez de apagá-los/regenerá-los automaticamente
RF-031	Exigir `IMPORT_ADOPTION` para entrada intermediária
RF-032	Suportar vocabulário baseline de estados locais

Anexo B — Registro epistemológico consolidado

ID	Classe	Enunciado vigente / registrado
E-001	USR	Primeira versão utilizável cobre da ideia ao deploy
E-002	USR	Fluxo principalmente baseado em BMAD estabilizado/customizado
E-003	USR	Usuário utilizará inicialmente sua própria IA
E-004	USR	Interface e experiência serão próprias do SB
E-005	USR	Existem dois modos conceituais: Guiado e Avançado
E-006	USR	Política padrão de autonomia será Supervisionada
E-007	USR	Autonomia varia conforme objetivo, risco e contexto
E-008	USR	Gestão de estado deve aproveitar BMAD quando adequado
E-009	USR	Assurance/auditabilidade são diferenciais centrais
E-010	USR	Segmentos 1–4 compõem o recorte inicial pretendido
E-011	PEND	Beachhead exato dentro de 1–4 ainda não fechado
E-012	REC histórico	Separar modo de interação de política de autonomia; posteriormente confirmado por D-018
E-013	PEND histórico	Avaliação de estado/handoff BMAD; posteriormente respondida pelo HO-SB-BMAD-001
E-014	PEND	Viabilidade técnica dos modos de conexão de IA precisa ser pesquisada

ID	Classe	Enunciado vigente / registrado
E-015	FAT	BMAD v6.12.0 possui mecanismos substanciais de persistência/readiness/tracking, porém distribuídos
E-016	FAT	Não foi demonstrado ledger transversal único BMAD para identity/autonomy/permissions/evidence/residual risk
E-017	FAT	Há casos reportados de divergência nativa e gaps de atribuição/resume
E-018	USR	Fronteira native-first confirmada
E-019	USR	Adapter versionado/anti-corruption layer confirmado
E-020	USR	Dashboards/status normalizados são projeções reconstruíveis
E-021	USR	MVP serial em escopos de conflito e preparado para concorrência futura
E-022	USR	Assurance Core obrigatório e graduado
E-023	USR	Modo de interação e autonomia são dimensões independentes
E-024	USR	Default: Guiado + Supervisionada
E-025	USR	Envelope funcional 1–4; beachhead comercial pode ser menor
E-026	USR	Assurance Core desde Ideação, com profundidade progressiva
E-027	USR	Um usuário principal por projeto no MVP
E-028	USR	Uma etapa macro primária ativa por vez
E-029	USR	Conclusão depende de Exit Contract
E-030	USR	Mudança upstream gera análise de impacto/staleness
E-031	USR	Entrada intermediária exige `IMPORT_ADOPTION`
E-032	USR	Vocabulário local baseline de estados confirmado

Anexo C — Pendências do ESTADO v0.6 e leitura reconciliada

ID	Registro no v0.6	Leitura vigente para este relatório
P-001	Confirmar envelope 1–4	**Resolvida por D-019** quanto ao envelope; beachhead exato continua aberto
P-002	Confirmar separação modo/autonomia	**Resolvida por D-018**
P-003	Definir alcance temporal de assurance	**Resolvida por D-020**
P-004	Individual vs equipe/workspace	**Resolvida por D-021**
P-005	Avaliar estado/handoff BMAD	Análise **concluída por HO-SB-BMAD-001** e arquitetura confirmada por D-014/D-015
P-006	Licença/uso/customização/distribuição BMAD	Aberta
P-007	Mecanismos de conexão às IAs	Aberta

ID	Registro no v0.6	Leitura vigente para este relatório
P-008	Salto/retorno/reabertura/invalidação	Parcialmente resolvida; detalhamento ainda aberto
P-009	Métricas/eventos	Adiável; aberta
P-010	Integrações repo/IDE/CI/CD/deploy	Adiável; aberta
P-011	Fronteira BMAD/SB	**Resolvida por D-014/D-015**
P-012	Runs paralelos	**Resolvida por D-016** para o MVP
P-013	Granularidade de provenance/assurance	Parcialmente resolvida; conteúdo mínimo do Core é decisão imediata
P-014	Inventário BMAD skill-by-skill	Aberta para etapa posterior

Anexo D — Fontes internas utilizadas

Este relatório foi produzido a partir de:

- `ESTADO_SystemBuilder_v0.6.md` — fonte canônica do produto na data deste relatório;
- `HO-SB-BMAD-001` / `RETORNO_PRODUCTMIND` — parecer especializado BMAD incorporado ao estado;
- histórico de decisões registrado nas versões 0.1 a 0.6 do ESTADO;
- rodada imediatamente posterior à v0.6, que formulou as quatro decisões descritas na Seção 14.

Nenhuma pesquisa de mercado ou validação externa adicional foi realizada para produzir este relatório.